1) a) When a TCP connection is established via a 3-way handshake, the transport layer keeps track of all the required information: & Source & destination IP's & ports. Hence, when send a TCP packet the OS already knows where to send it.

With UDP however, no such information is stored. Hence you need to specify to the OS where you are sending the packet.

b) A UDP socket is fully identified by the a tuple consisting of a the destination IP & destination port.

A TCP socket is identified by the Sorce IP, Sorce port, Destinati IP & destination port.

c) Yes, as the transport layer will still identify the port 8080 hostA and delivers the segment to that socket regardless of who sent it.

To process at host A can determine the who sent the segment by looking at the soure IP & port of the incoming segment.

2) a) Although both use TCP connections, in HTTP 1.0 these connections are not persistent. ~~Instead~~ They are established & closed after every request.

HTTP 1.1 uses persistent connections allowing multiple requests ~~after~~ over a/ connection.

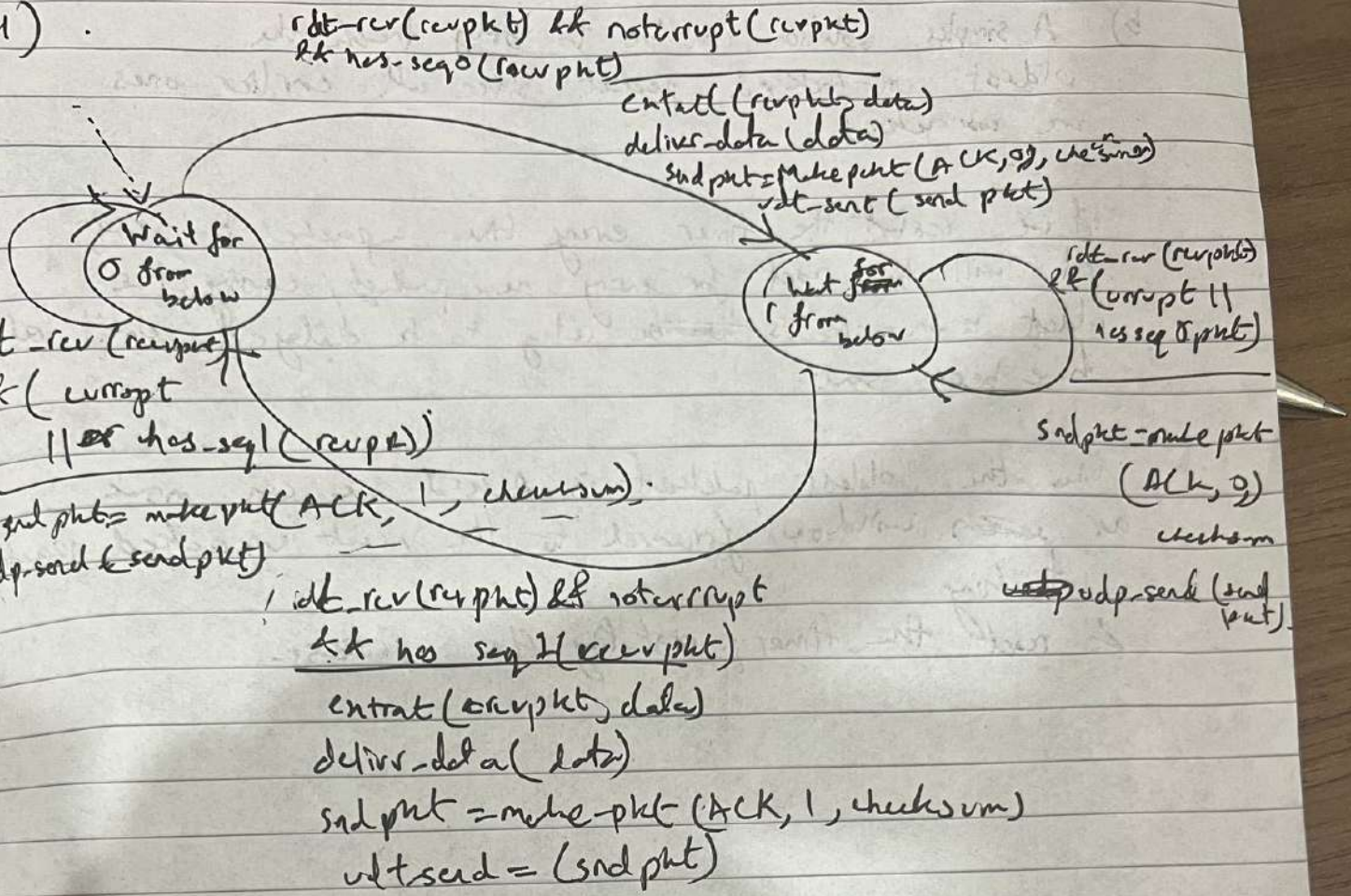
3) If addressed:
- Inefficient use of a connection, establishing a new one for every request.

- Inability to pipeline, each request has to be completed before a subsequent one can be made.

c) ~~The~~ Abot of time is saved by avoiding the need to do a 3-way handshake for every request leading to faster pageloads, less CPU load & more efficient communication. The ability to pipeline makes that even faster as requests can be made simultaneously.

3) Application designers can exploit TCP by opening multiple parallel TCP connections. Allowing their application to grab more bandwidth at the expense of other dataflows.

4) .



5) A potential solution is to use a min-heap to track the timeout times for the sent packets. Maintain a single timer to track the ~~soonest~~ earliest timeout. On expiry, process packets that have expired & resend them, reset their timers & use the hardware timer for the next earliest expiry.

a) DS:- Min-heap or some sorted structure
~~Maybe a hashmap to track associate timers & packet ID~~

Computation:- Insert/delete on send/ack/expiry, recalculation of timer.

Memory:- Store timer for each unacked packet.

b) A simpler solution can be to only track the oldest un-Ack'ed packet since all earlier ones are acked.

If we restart the timer every time a packet is sent, rt will be reset for every new packet, causing the oldest unacked's ~~be~~ retry to be delayed until all have been sent.

When the oldest packet is Acked we can move our ~~send~~ window forward to the next unacked packet tracking & reset the timer, restarting the process.